PROJECT AND TEAM INFORMATION

Project Title

(Try to choose a catchy title. Max 20 words).

DocuCompiler: A Compiler-Inspired Intelligent Document Summarization System

Student / Team Information

Team Name:	***hydrangea***
Team member 1 (Team Lead) *Name: Paras Rawal* *Student Id: 230111703* *Email: parasrawal117@gmail.com* *University roll no: 2319207*	
Team member 2 *Name: Ishan pant* *Student Id: 230111630* *Email: ishanpant57@gmail.com* *University roll no: 2318894*	
Team member 3 *Name: Paras pant* *Student Id: 23011046* *Email: paraspant181@gmail.com* *University roll no: 2319208*	
Team member 4 *Name: Shobhit joshi* *Student Id: 23011783* *Email: joshishobhit1705@gmail.com* *University roll no: 2319583*	

PROPOSAL DESCRIPTION (10 pts)

Motivation (1 pt)

(Describe the problem you want to solve and why it is important. Max 300 words).

Modern documents such as research papers, reports, legal files, and technical manuals are often long and difficult to process quickly. Users need concise summaries and quick answers without reading entire documents. Existing summarization tools rely heavily on large transformer models that demand high computational resources and GPUs.

This project proposes a ***compiler-inspired document processing system*** *that treats a document like source code and processes it through structured phases such as lexical analysis, parsing, semantic analysis, and intermediate representation before generating a summarized "output program." This approach makes the system modular, efficient, and suitable for low-resource environments.*

By applying compiler design principles to Natural Language Processing, the project demonstrates how classical computing concepts can be adapted to solve modern AI problems in a structured and optimized manner. The system will be lightweight, CPU-friendly, and capable of summarizing and extracting insights from large documents effectively.

State of the Art / Current solution (1 pt)

(Describe how the problem is solved today (if it is). Max 200 words).

Most current document summarization systems depend on large deep learning models such as BERT, GPT, or large T5 variants. While they provide good results, they are computationally expensive and unsuitable for low-end systems. Many tools also lack a structured processing pipeline and treat summarization as a single-step black-box process.

In contrast, this project introduces a ***multi-phase structured pipeline inspired by compilers****, where each phase transforms the document representation step by step. This not only improves clarity and modularity but also allows optimization at each stage, similar to how compilers optimize code before execution.*

Project Goals and Milestones (2 pts)

(Describe the project general goals. Include initial milestones as well any other milestones. Max 300 words).

Main Goals

- *Design a compiler-style pipeline for document processing*
- *Implement lexical and syntactic style processing for text*
- *Generate structured intermediate representations of documents*
- *Produce concise summaries using extractive and lightweight abstractive techniques*
- *Enable document-based question answering using semantic retrieval*

Milestones

Week	***Milestone***
Week 1–2	***Requirement analysis and compiler-model mapping***
Week 3	***Design document processing pipeline (Lexical, Syntax, Semantic phases)***
Week 4	***Implement text extraction and tokenization (Lexical Analysis)***
Week 5	***Sentence structure modeling and chunking (Syntax Analysis)***

Week 6	***Meaning extraction, keyword detection (Semantic Analysis)***
Week 7	***Intermediate Representation and summary generation***
Week 8	***Question-answer module integration***
Week 9	***Frontend interface development***

.

Project Approach (3 pts)

(Describe how you plan to articulate and design a solution. Including platforms and technologies that you will use. Max 300 words).

Compiler-Based System Design

This project models document processing similar to a compiler pipeline:

Compiler Phase	***Equivalent NLP Task***	***Tools/Methods***
Lexical Analysis	*Tokenization, cleaning, stopword removal*	*NLTK / SpaCy*
Syntax Analysis	*Sentence segmentation, structure modeling*	*SpaCy / rule-based parsing*
Semantic Analysis	*Keyword extraction, topic detection*	*TF-IDF, TextRank*
Intermediate Representation	*Chunk embeddings / sentence vectors*	*Sentence Transformers*
Optimization	*Redundancy removal, sentence ranking*	*Similarity scoring*
Code Generation	*Final summary creation*	*TextRank + T5-small*

Technologies Used

Component	***Technology***	***Purpose***
Backend	*Python (FastAPI)*	*API and pipeline control*
NLP Processing	*NLTK, SpaCy, Transformers*	*Text analysis*
Summarization	*TextRank + T5-small*	*Lightweight summarization*
Embeddings	*all-MiniLM-L6-v2*	*Semantic representation*
Vector Storage	*FAISS*	*Fast retrieval*
Database	*SQLite*	*Store documents, summaries, history*
Frontend	*HTML, CSS, JS / React*	*User interaction*

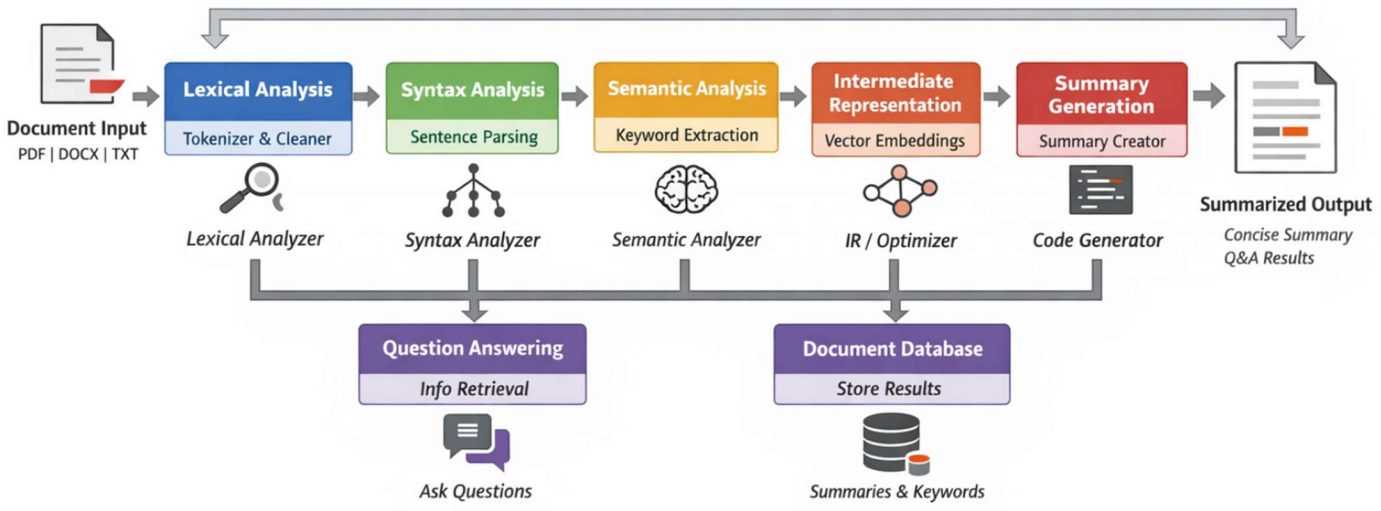
System Workflow

1. *User uploads a document*
2. *Text is extracted and passed through* ***Lexical Analysis***
3. *Cleaned tokens are grouped into sentences (**Syntax Phase**)*
4. *Important terms and meanings identified (**Semantic Phase**)*
5. *Sentences converted into embeddings (**Intermediate Representation**)*

6. *Ranking and optimization remove redundancy*
7. *Final summary generated*
8. *User can ask questions; relevant chunks retrieved via FAISS*

System Architecture (Entity Relationship Diagram)(2 pts)

(Provide an overview of the system, identifying its main components and interfaces in the form of a diagram using a tool of your choice).



Project Outcome / Deliverables (1 pts)

(Describe what are the outcomes / deliverables of the project. Max 200 words).

- *A working compiler-inspired NLP pipeline*
- *Document summarization system for PDFs, DOCX, and TXT files*
- *Keyword extraction and topic detection module*
- *Retrieval-based document question answering feature*
- *Web interface for document upload and interaction*
- *Documentation including system design, pipeline explanation, and architecture diagrams*
- *This project demonstrates the practical application of compiler design concepts beyond programming languages and into intelligent document processing.*

Assumptions

(Describe the assumptions (if any) you are making to solve the problem. Max 100 words)

- *Users upload text-based documents in supported formats*
- *Documents are primarily in English*
- *System runs on CPU-only environments*
- *Internet is not required for processing once models are downloaded*

References

(Provide a list of resources or references you utilised for the completion of this deliverable. You may provide links).

1. ***Phases of a Compiler (lexical, syntax, semantic, etc.)***
   https://www.geeksforgeeks.org/compiler-design/phases-of-a-compiler/
2. ***Compiler Design Tutorial (basic & advanced topics)***
   https://www.tutorialspoint.com/compiler_design/index.htm
3. ***Slide Overview of Compiler Design & Phases***
   https://www.slideshare.net/slideshow/introduction-to-compiler-design-and-phases-of-compiler/270595653

Text Summarization & NLP References (Methodology)

4. ***Text Summarization in NLP (extractive vs abstractive)***
   https://www.geeksforgeeks.org/nlp/text-summarization-in-nlp/
5. ***TextRank Explained (graph based summarization)***
   https://blog.fastforwardlabs.com/2021/09/22/automatic-summarization-from-textrank-to-transformers.html
6. ***Text Summarization Using TextRank (tutorial with implementation)***
   https://www.analyticsvidhya.com/blog/2018/11/introduction-text-summarization-textrank-python/
7. ***Academic Study: Document Summarization with TextRank***
   https://www.researchgate.net/publication/320855773_Document_Summarization_using_TextRank_and_Semantic_Network